Яндекс

Как Java-фреймворки ускоряются на примере Quarkus



Андрей Кулешов

Руководитель отдела разработки,
Yandex Infrastructure



$ whoami?



Андрей Кулешов

Руководитель отдела разработки
SourceCraft Security,
Yandex Infrastructure

akuleshov7_live

$ whoami?



Андрей Кулешов

Руководитель отдела разработки
SourceCraft Security,
Yandex Infrastructure

JPoint podlodka Joker<?> Code Fest

akuleshov7_live

$ whoami?



Андрей Кулешов

Руководитель отдела разработки
SourceCraft Security,
Yandex Infrastructure

JPoint podlodka Joker<?> Code Fest

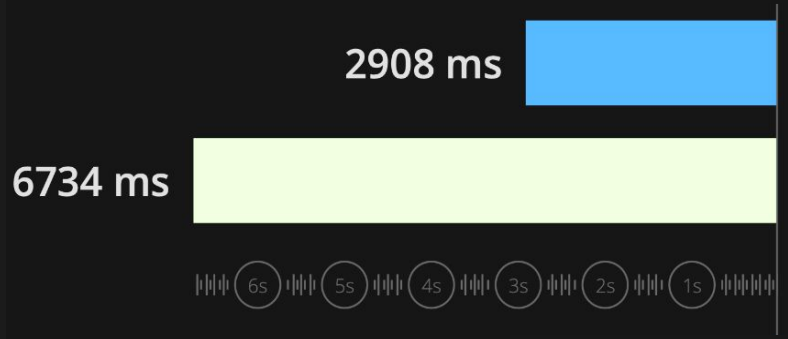
intel Deutsche Bank HUAWEI Я

akuleshov7_live

СРАЗУ ЦИФРЫ

Boot + First Response Time

(Shorter is better)

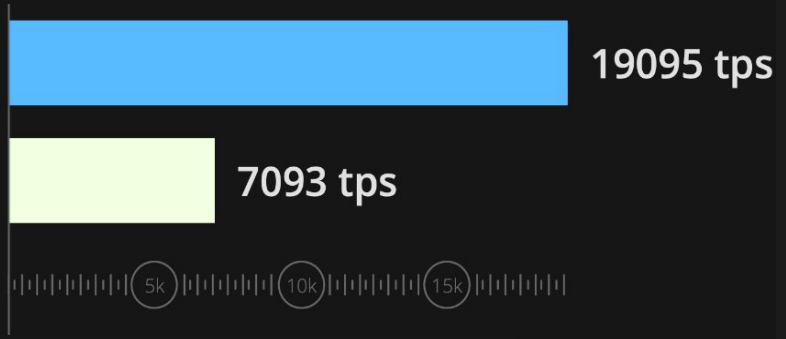


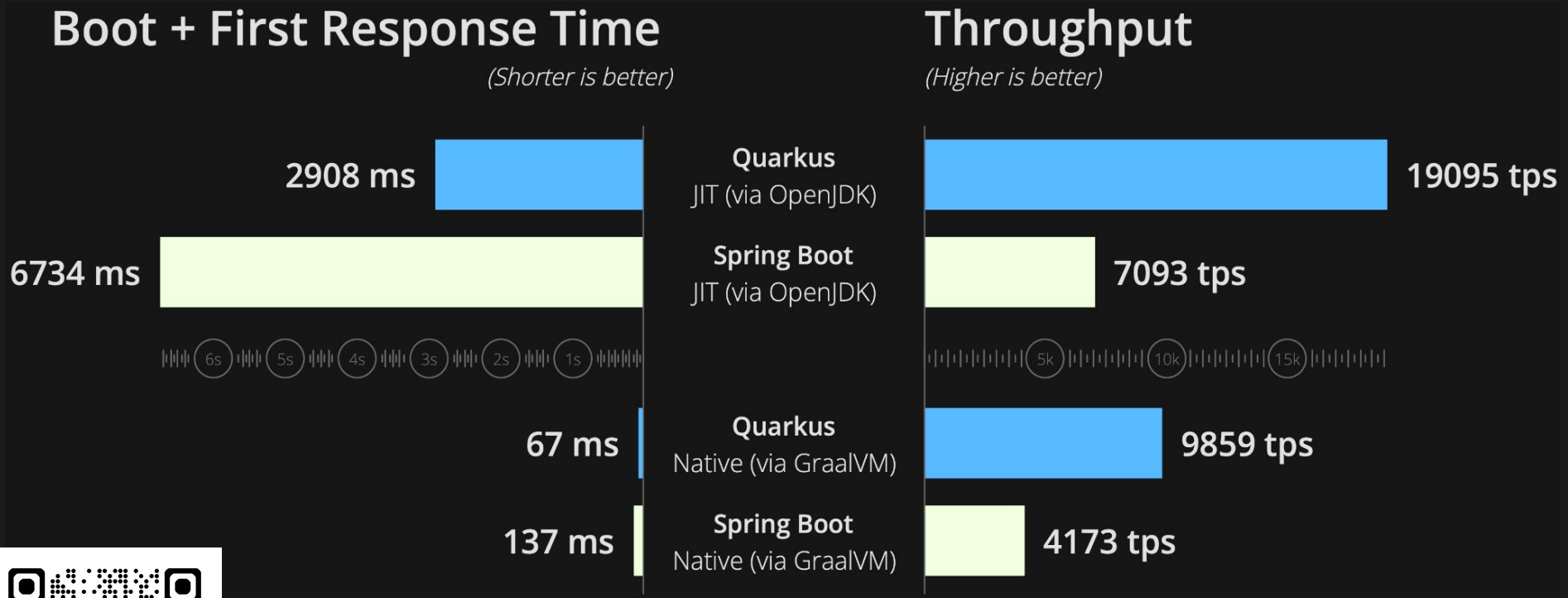
Quarkus
JIT (via OpenJDK)

Spring Boot
JIT (via OpenJDK)

Throughput

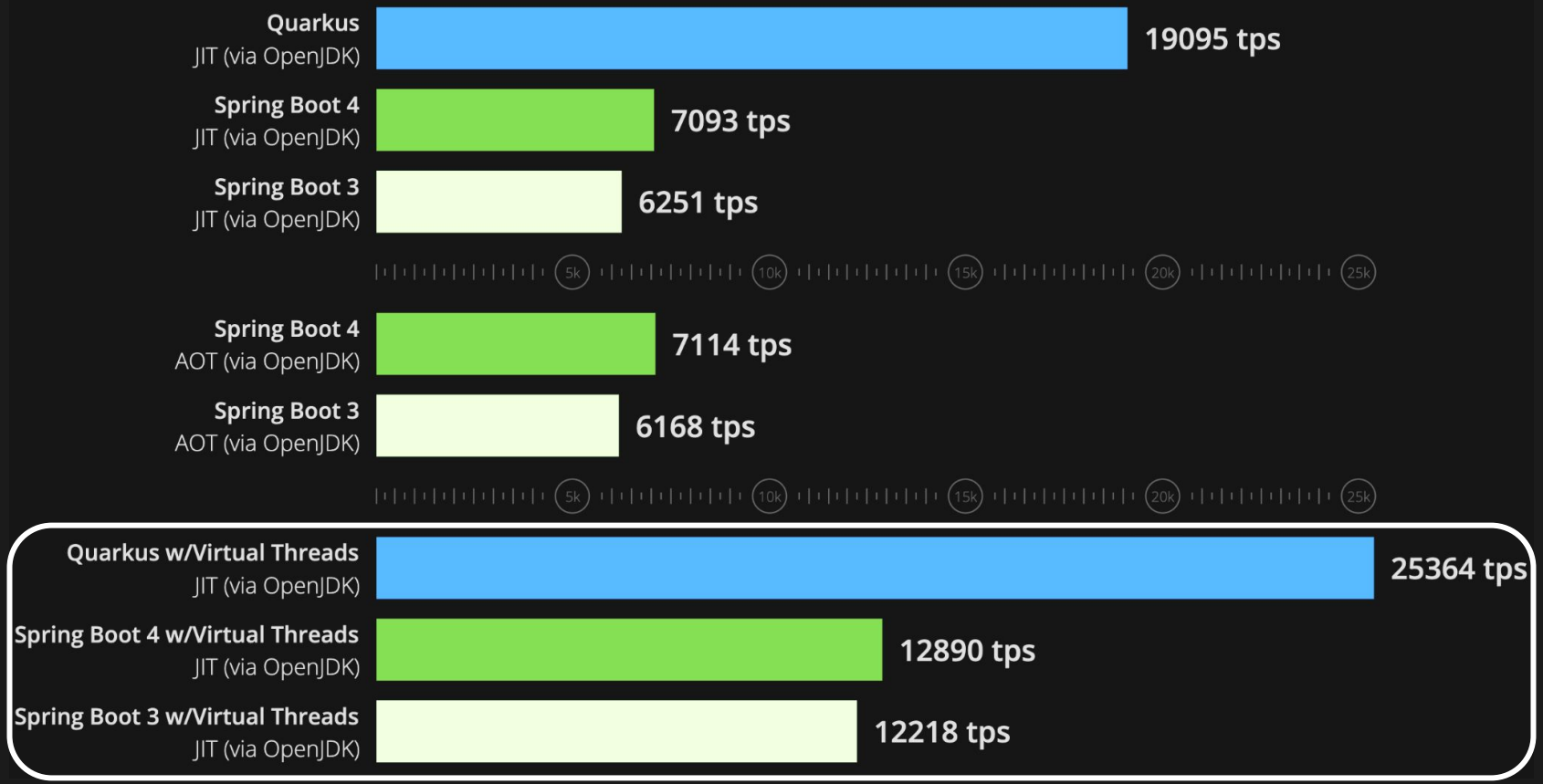
(Higher is better)



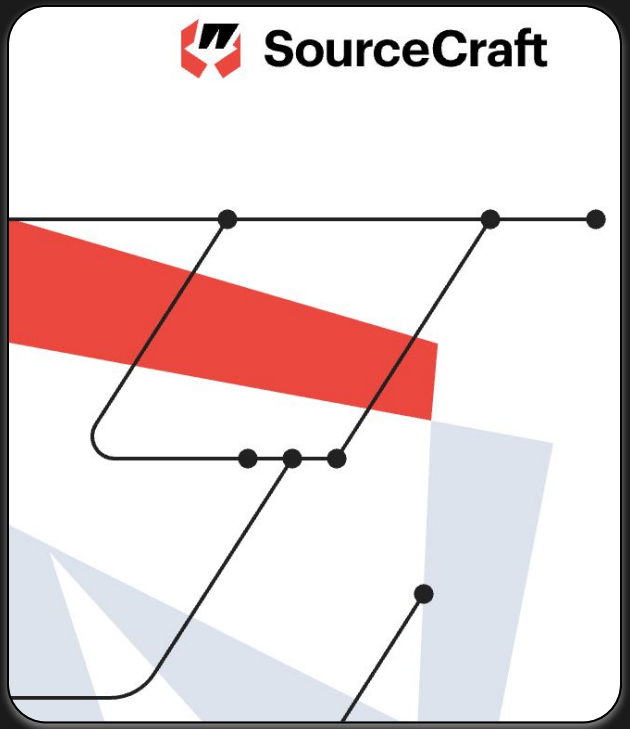

Boot + First Response Time
(Shorter is better)
Throughput
(Higher is better)
2908 ms
Quarkus
JIT (via OpenJDK)
19095 tps
6734 ms
Spring Boot
JIT (via OpenJDK)
7093 tps
6s
5s
4s
3s
2s
1s
5k
10k
15k
67 ms
Quarkus
Native (via GraalVM)
9859 tps
137 ms
Spring Boot
Native (via GraalVM)
4173 tps

Throughput

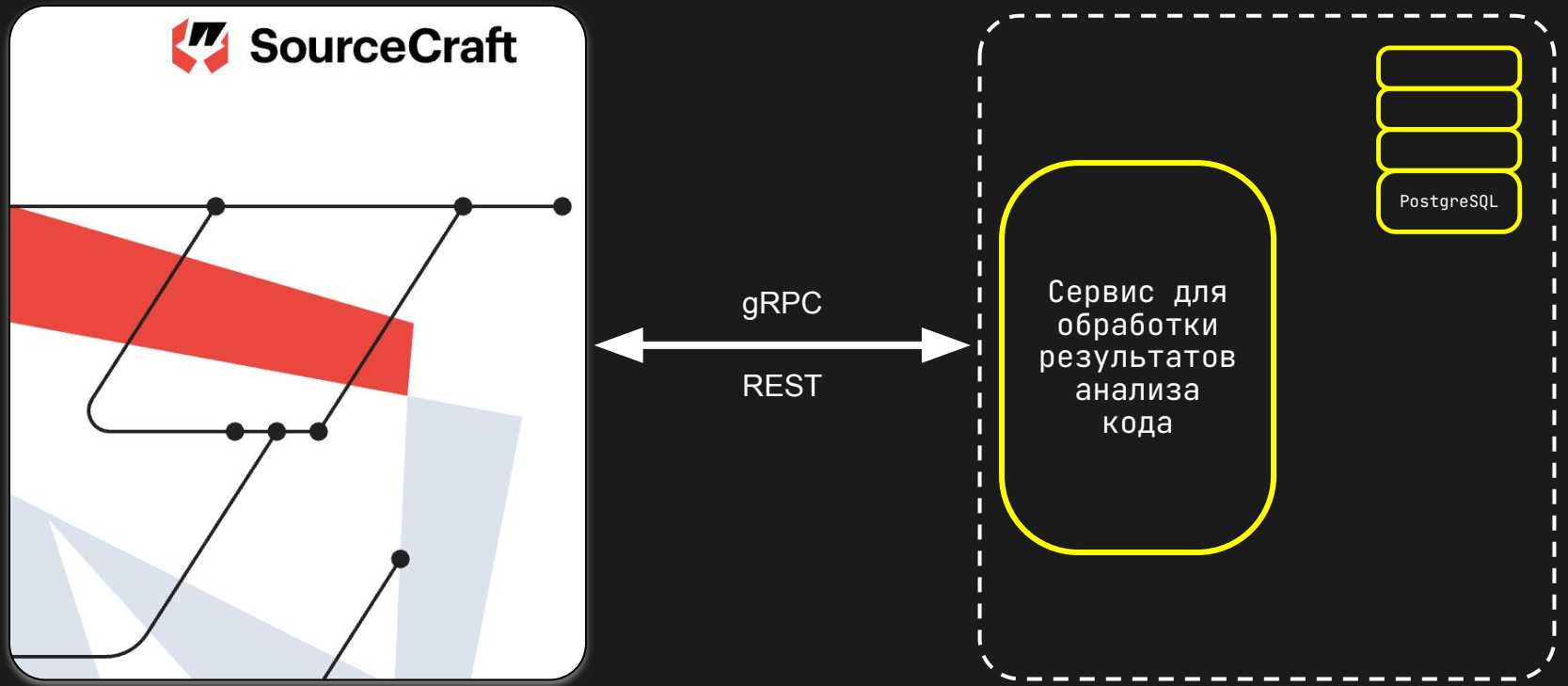
(Higher is better)



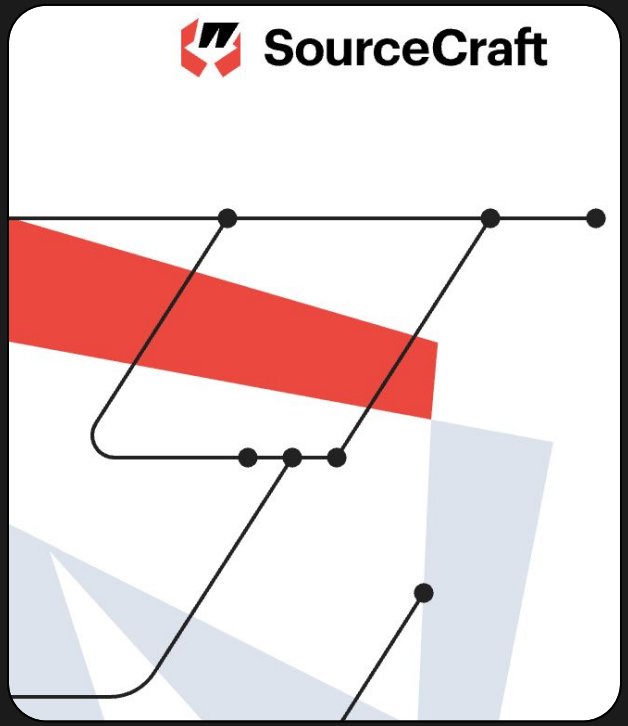
Моя история: PROD



Моя история: PROD



Моя история: PROD



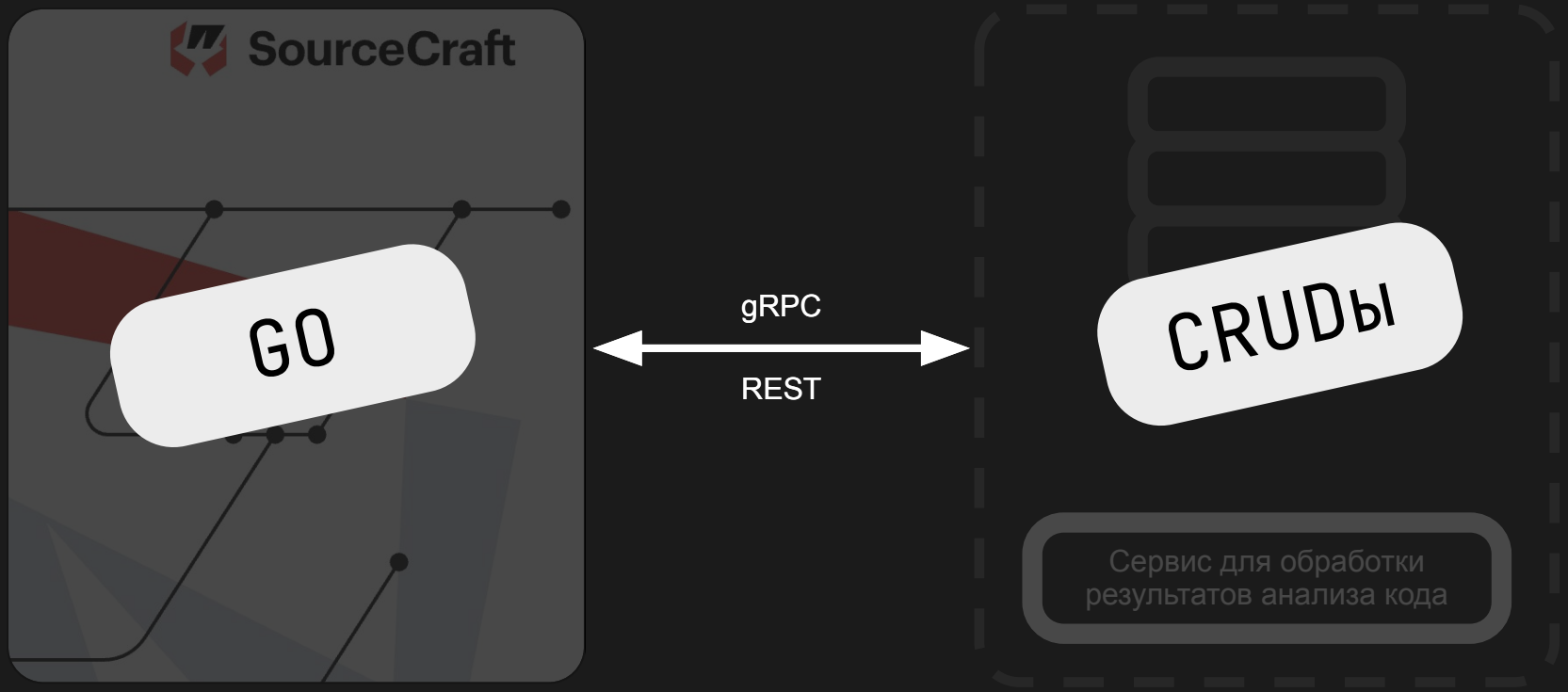
gRPC

REST

CRUDы

Сервис для обработки результатов анализа кода

Моя история: PROD

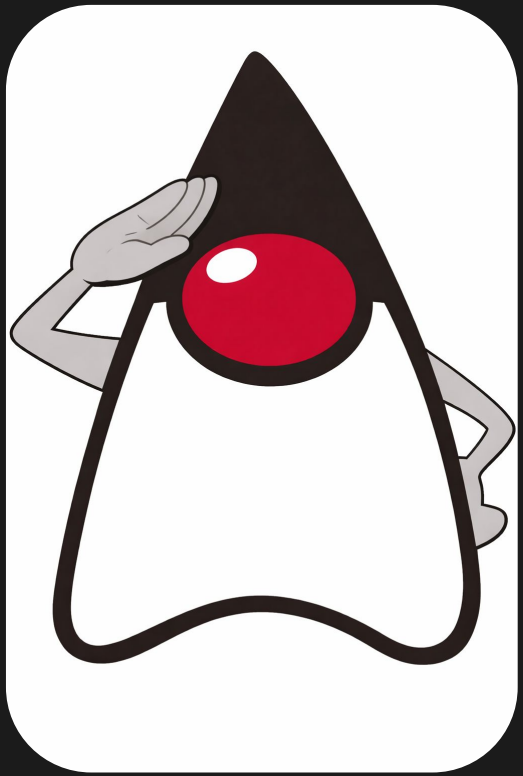


Моя история: PROD



Вопрос чести жависта!





По итогу:

- Год крутим небольшой сервис на Quarkus JVM в PROD

По итогу:

- Год крутим небольшой сервис на Quarkus JVM в PROD
- 50k строк, 60 grpc и десяток REST эндпоинтов

По итогу:

- Год крутим небольшой сервис на Quarkus JVM в PROD
- 50k строк, 60 grpc и десяток REST эндпоинтов
- 0 инцидентов на стороне фреймворка

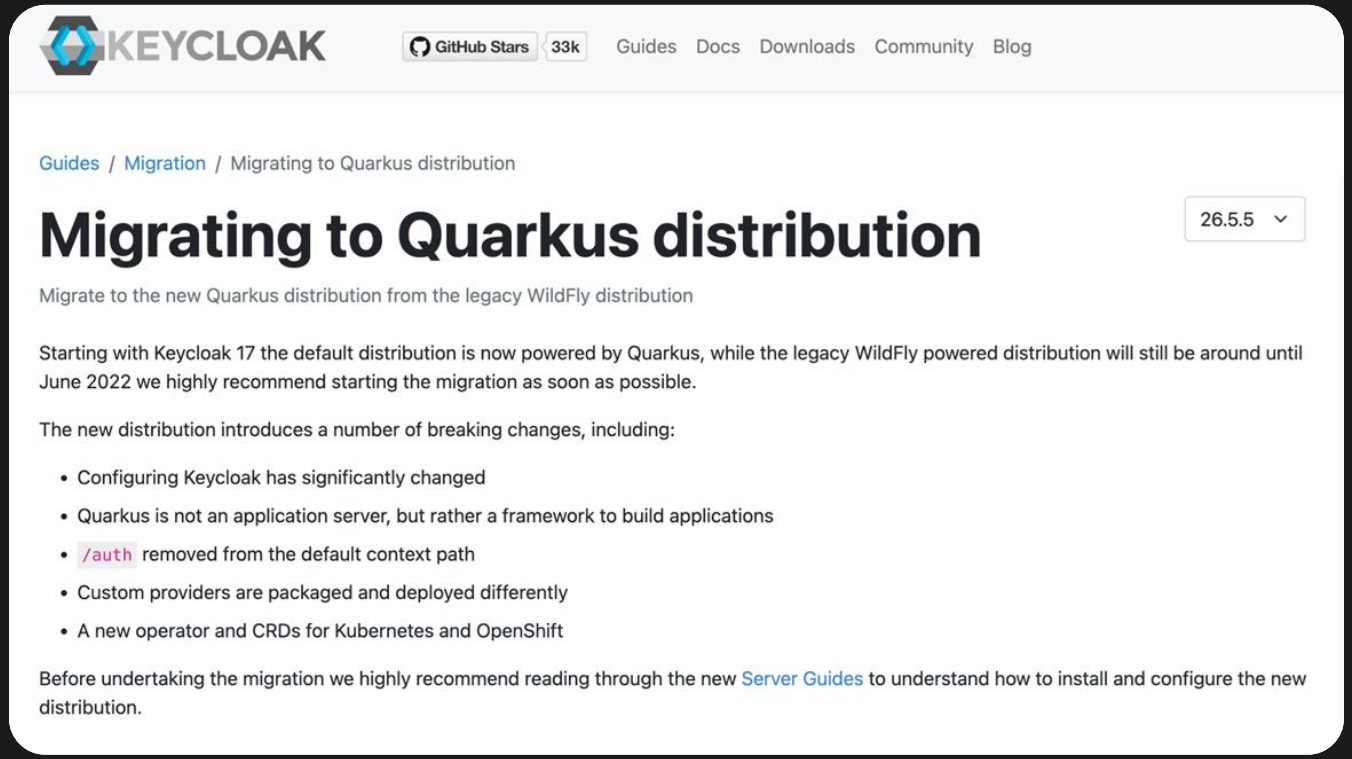


По итогу:

- Год крутим небольшой сервис на Quarkus JVM в PROD
- 50k строк, 60 grpc и десяток REST эндпоинтов
- 0 инцидентов на стороне фреймворка
- ~3 секунды на честный старт (не lazy)

Кто использует Quarkus?

Кто использует Keycloak?



Как всё начиналось?



Continuously Measuring, Continuously Improving

Quarkus is dedicated to continuously improving performance, especially for code running on the critical execution (hot) path. Through ongoing optimizations, Quarkus ensures that every instruction and allocated byte matters, making it one of the most efficient frameworks available for developing high-performance, cloud-ready applications.

Continuously Measuring, Continuously Improving

Quarkus is dedicated to continuously improving performance, especially for code running on the critical execution (hot) path. Through ongoing optimizations, Quarkus ensures that every instruction and allocated byte matters, making it one of the most efficient frameworks available for developing high-performance, cloud-ready applications.



Куда ведет ссылка?

Continuously Measuring, Continuously Improving

Quarkus is dedicated to continuously improving performance, especially for code running on the critical execution (hot) path. Through ongoing optimizations, Quarkus ensures that every instruction and allocated byte matters, making it one of the most efficient frameworks available for developing high-performance, cloud-ready applications.



Куда ведет ссылка?

Правильно, не лучший TechEmpower

Continuously Measuring, Continuously Improving

Quarkus is dedicated to continuously improving performance, especially for code running on the critical execution (hot) path. Through ongoing optimizations, Quarkus ensures that every instruction and allocated byte matters, making it one of the most efficient frameworks available for developing high-performance, cloud-ready applications.



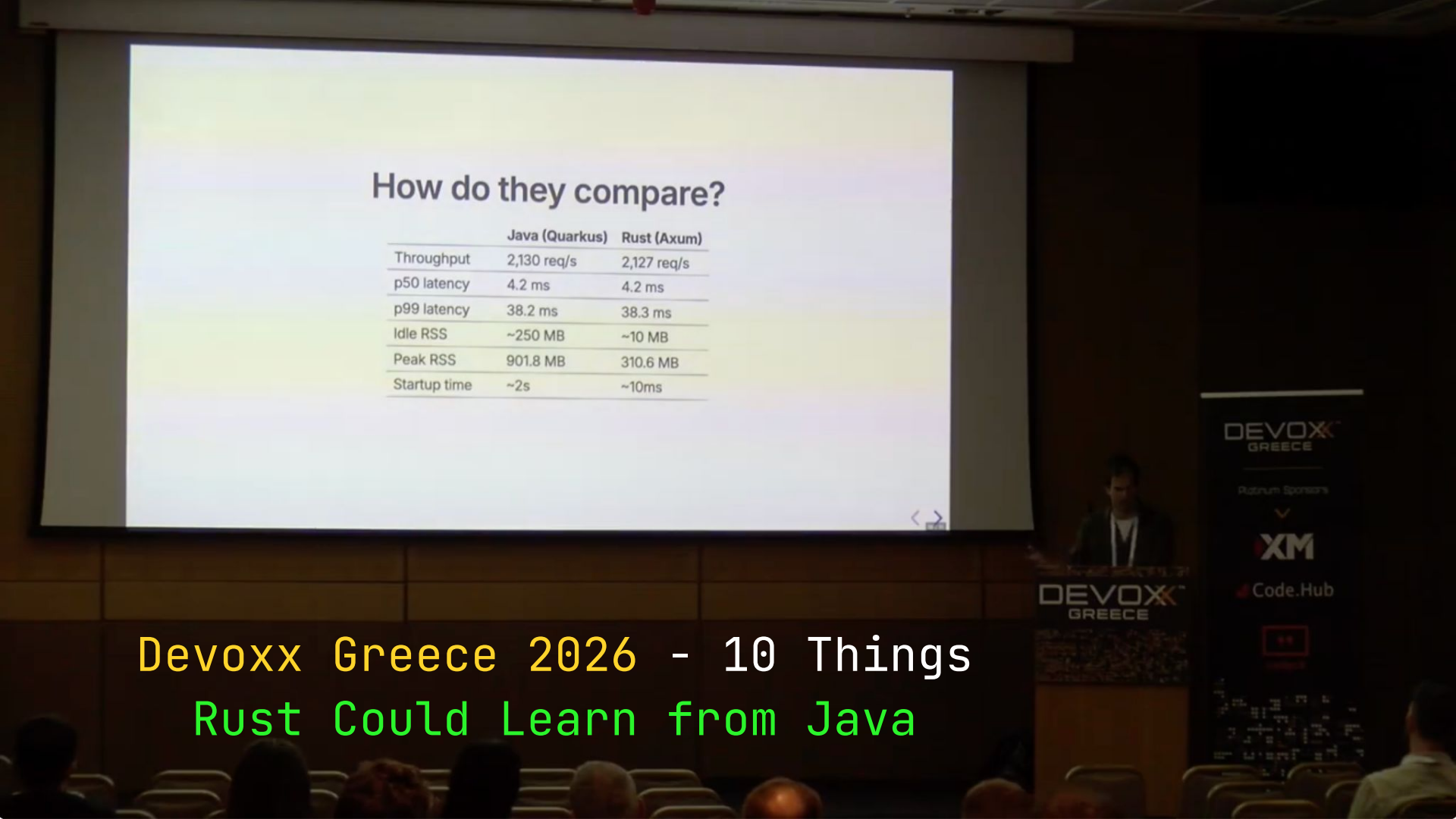
Куда ведет ссылка?

Правильно, не лучший TechEmpower

Our intention with the benchmark was to replicate the experience of a normal user, not to tune each framework to within an inch of its life. (There's TechEmpower for that.)



Что люди говорят?



Devoxx Greece 2026 - 10 Things Rust Could Learn from Java

Нам обещают

Fast startup times

High throughput

Low response latency

Reduced memory footprint and
minimal resource consumption

Нам обещают

Fast startup times

High throughput

Low response latency

Reduced memory footprint and
minimal resource consumption

Быстрый старт
почему так?


GraalVM™
Build faster, smaller, leaner applications


GraalVM™
Build faster, smaller, leaner applications

GraalVM умирает? 😱

r/java • -6 мес.
mikebmx1

Detaching GraalVM from the Java Ecosystem Train

blogs.oracle.com Открыть

Пост в архиве. Нельзя добавлять новые комментарии и нельзя ставить оценки.

79 75 Поделиться

Сортировка по: Лучшие Найти комментарии

thomaswue • 6 мес. назад

Hi, GraalVM project lead at Oracle here. Don't worry, we are not going anywhere and we will continue our mission to run programs faster and more efficient :).

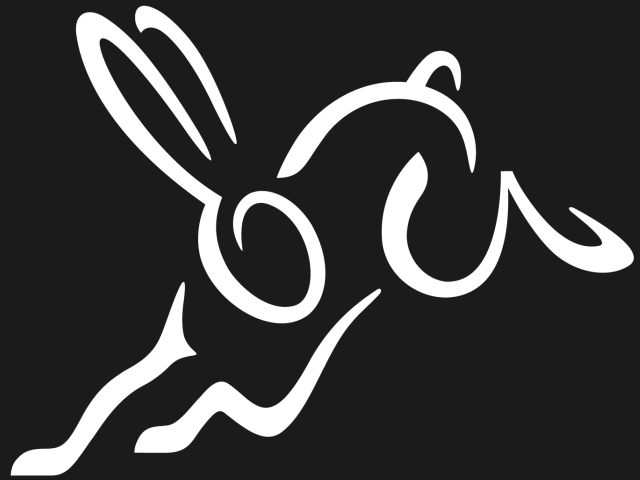
82 Награда Поделиться ···

dmigowski • 6 мес. назад

The blog post sounds like GraalVM will vanish from the OpenJDKs and that you have to be a paying customer of Oracle to continue using it with newer JDK releases.

Could you clarify?

EDIT: This is from r/quarkus:



Но почему тогда старт у Quarkus даже без нативного исполнения быстрее?

Классический подход

Обычное Java-приложение
при старте на runtime:



Классический подход

Обычное Java-приложение
при старте на runtime:

- Автоконфигурация, загрузка конфигураций



Классический подход

Обычное Java-приложение
при старте на runtime:

- Автоконфигурация, загрузка конфигураций
- Сканирование бинов/классов



Классический подход

Обычное Java-приложение
при старте на runtime:

- Автоконфигурация, загрузка конфигураций
- Сканирование бинов/классов
- Dependency Injection с рефлексией



Классический подход

Обычное Java-приложение
при старте на runtime:

- Автоконфигурация, загрузка конфигураций
- Сканирование бинов/классов
- Dependency Injection с рефлексией
- Работа с приватными полями
  setAccessible(true)



Quarkus: Стартуем быстро, делая меньше

- Анализ и подготовка на этапе сборки (codegen). Берем только нужное

Quarkus: Стартуем быстро, делая меньше

- Анализ и подготовка на этапе сборки (codegen). Берем только нужное
- Максимум работы переносится с runtime на build-time

Quarkus: Стартуем быстро, делая меньше

- Анализ и подготовка на этапе сборки (codegen). Берем только нужное
- Максимум работы переносится с runtime на build-time
- Дорогие операции выполняются один раз при сборке, а не при каждом старте

Quarkus: Стартуем быстро, делая меньше

- Анализ и подготовка на этапе сборки (codegen). Берем только нужное
- Максимум работы переносится с runtime на build-time
- Дорогие операции выполняются один раз при сборке, а не при каждом старте
- Предсказуемый быстрый старт, компактность и эффективность

Quarkus: Стартуем быстро, делая меньше?

Интересное по поводу замеров

❗ Проблема ленивой инициализации

- Приложение может:
  - быстро запуститься, написать started
  - но долго обрабатывать первый запрос

❗ Проблема ленивой инициализации

- Приложение может:
  - быстро запуститься, написать started
  - но долго обрабатывать первый запрос

- Поскольку часть инициализации происходит:
  - при первом HTTP-запросе
  - при первом обращении к DI / REST контроллеру / сериализации

В тесте фиксируется первый запрос

```
@Path("/")
public class GreetingEndpoint {


    private static final String template = "Hello, %s!";


    @GET
    @Path("/greeting")
    @Produces(MediaType.APPLICATION_JSON)
    public Greeting greeting(@QueryParam("name") String name) {
          System.out.println(
              new SimpleDateFormat("HH:mm:ss.SSS")
                  .format(new Date())
          );
        String suffix = name != null ? name : "World";
        return new Greeting(String.format(template, suffix));
    }

}
```

В тесте фиксируется первый запрос

```
@Path("/")
public class GreetingEndpoint {


   private static final String template = "Hello, %s!";


   @GET
   @Path("/greeting")
   @Produces(MediaType.APPLICATION_JSON)
   public Greeting greeting(@QueryParam("name") String name) { ... }

   void onStart (@Observes StartupEvent startup) {
         System.out.println(
              new SimpleDateFormat("HH:mm:ss.SSS")
                   .format(new Date())
         );
   }

}
```

Это позволяет увидеть реальный старт:

```
$ while [[ "$(curl -s -o /dev/null -w ''%{http_code}''
localhost:8080/api/greeting)" ≠ "200" ]]; do sleep .00001; done
```

Это позволяет увидеть реальный старт:

```
$ while [[ "$(curl -s -o /dev/null -w ''%{http_code}''
localhost:8080/api/greeting)" ≠ "200" ]]; do sleep .00001; done
```

```
$ date +"%T.%3N" &&  ./target/quarkus-timing-runner

10:57:32.508 время старта скрипта
10:57:32.512 время StartupEvent
2026-03-20 10:57:32,512 INFO  [io.quarkus] (main) Quarkus 0.11.0 started in
0.002s. Listening on: http://127.0.0.1:8080
2026-03-20 10:57:32,512 INFO  [io.quarkus] (main) Installed features: [cdi,
rest, rest-jackson]
10:57:32.537 время обслуживания первого запроса
```

Что Quarkus делает на этапе сборки

- Читает часть конфигурации приложения

Что Quarkus делает на этапе сборки

- Читает часть конфигурации приложения
- Сканирует classpath и аннотации

Что Quarkus делает на этапе сборки

- Читает часть конфигурации приложения
- Сканирует classpath и аннотации
- Строит модель и индексы для приложения

Что Quarkus делает на этапе сборки

- Читает часть конфигурации приложения
- Сканирует classpath и аннотации
- Строит модель и индексы для приложения
- Генерирует код и вмешивается в байткод → точные инструкции запуска

Что Quarkus делает на этапе сборки

- Читает часть конфигурации приложения
- Сканирует classpath и аннотации
- Строит модель и индексы для приложения
- Генерирует код и вмешивается в байткод → точные инструкции запуска
- Определяет, какие компоненты реально нужны → лишний код удаляется

Что Quarkus делает на этапе сборки

- Читает часть конфигурации при
- Сканирует
- ия
- од → точные инструкции запуска
- ементы реально нужны → неиспользуемые
  ементы и лишний код удаляются

Всё, что можно вычислить заранее - вычисляется на compile/build-time

Compile/Build-Time



Compile/Build-Time

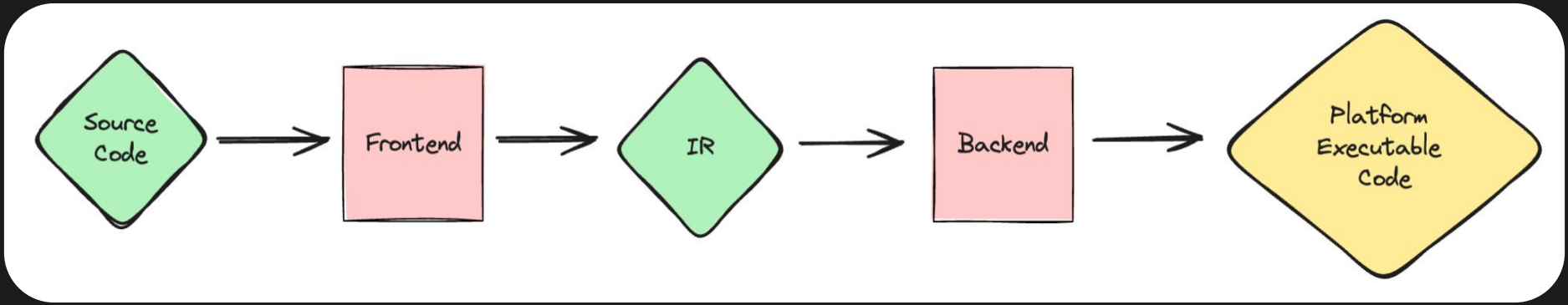
```
public class
QuarkusCompiler
```



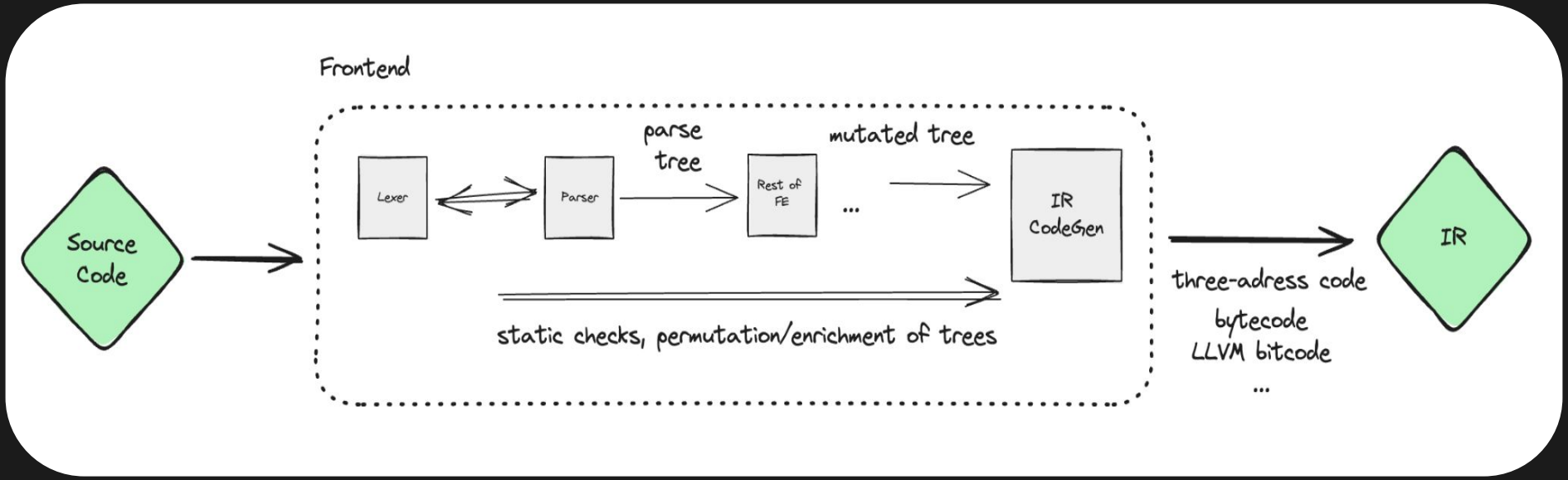
Значит, будет полезно немного погрузиться в компиляторный мир



Типичный процессинг компилятора

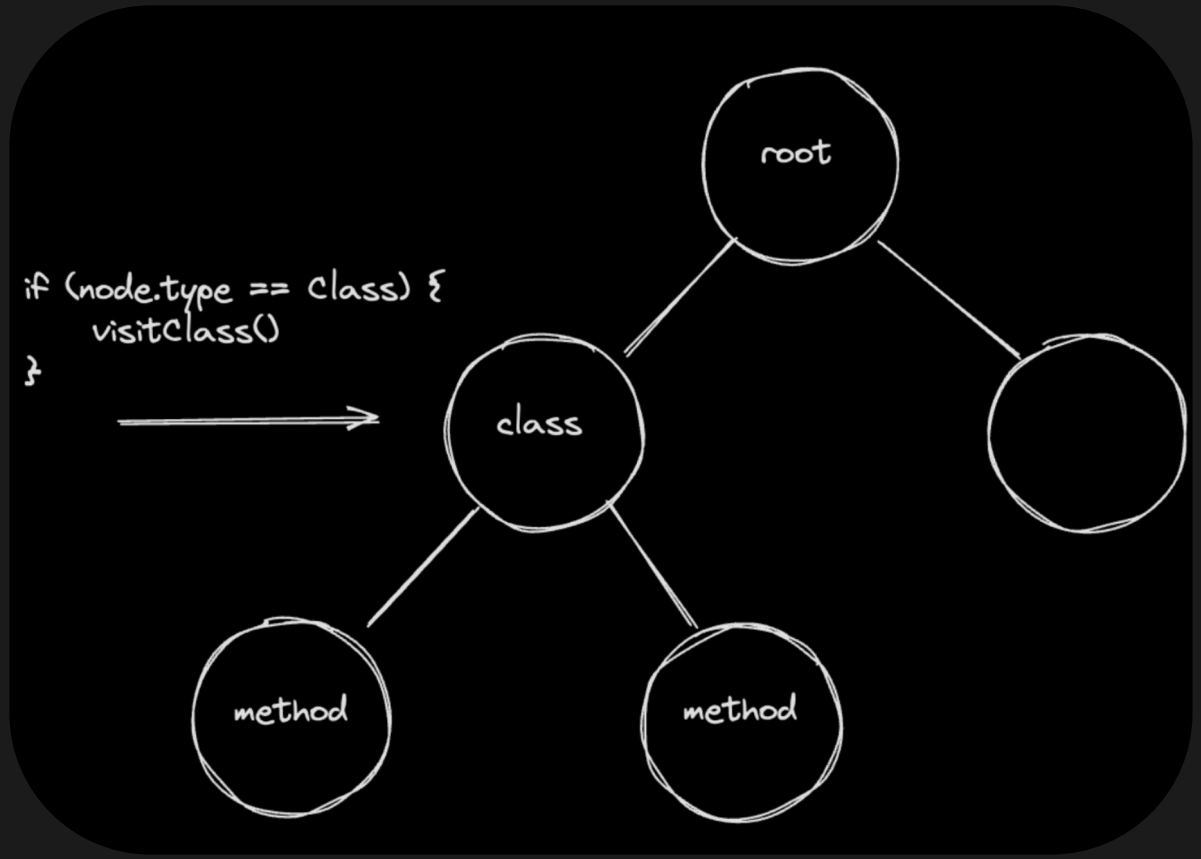


Типичный процессинг компилятора



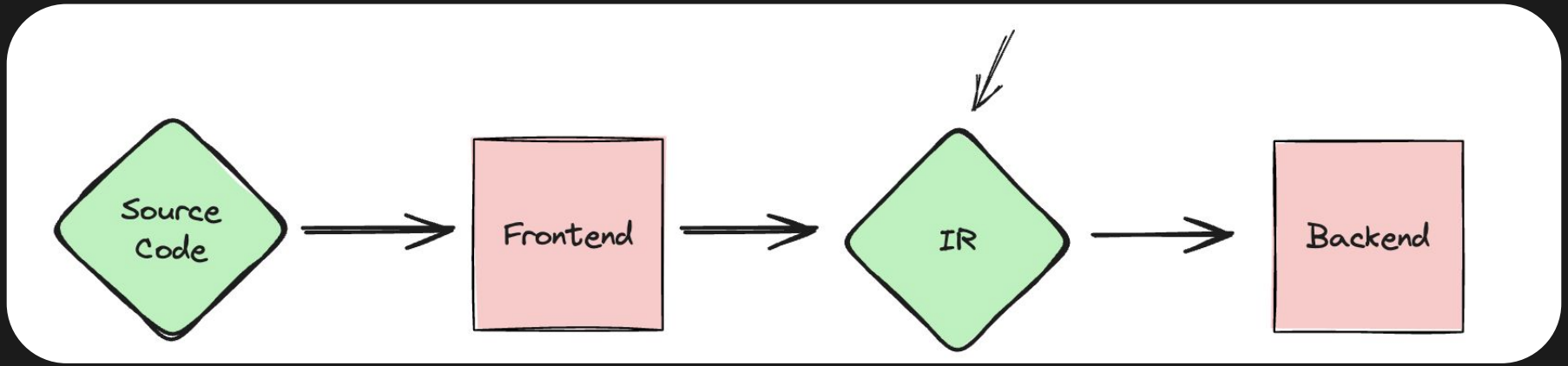
мутируют и обогащаются AST

Вмешательство на разборе исходного кода



Можно вмешиваться в AST на уровне фронтенда, а можно в IR

Наш bytecode



Вмешаться в bytecode (IR)!

Кем используется?

- the OpenJDK, to generate the lambda call sites, and also in the Nashorn compiler;
- the Groovy compiler and the Kotlin compiler;
- Cobertura and Jacoco, to instrument classes in order to measure code coverage;
- Byte Buddy, to dynamically generate classes, itself;
- Used in other projects such as Mockito (to generate mock classes);
- Gradle, to generate some classes at runtime.



Вмешаться в bytecode (IR)!

Кем используется?

JEP 484: Class-File API

- ~~the OpenJDK, to generate the lambda call sites, and also in the Nashorn compiler;~~
- the Groovy compiler and the Kotlin compiler;
- Cobertura and Jacoco, to instrument classes in order to measure code coverage;
- Byte Buddy, to dynamically generate classes, itself;
- Used in other projects such as Mockito (to generate mock classes);
- Gradle, to generate some classes at runtime.



Вмешаться в bytecode (IR)!

Кем используется?

JEP 484: Class-File API

- the OpenJDK, to generate the lambda call

JEP 484 Goals

- Provide an **API for processing class files** that tracks the class file format defined by the Java Virtual Machine Specification.
- Enable JDK components to migrate to the standard API, and eventually remove the JDK's internal copy **of the third-party ASM library.**

- Gradle, to generate some classes at runtime.



Удалим поле **removeMe** из Example

```
class Example {
    int keepMe;
    String removeMe;
}
```



```java
public class RemoveFieldClassVisitor extends ClassVisitor {

  public RemoveFieldClassVisitor(ClassVisitor cv) {
      super(Opcodes.ASM9, cv);
  }

  @Override
  public FieldVisitor visitField(
      int access,
      String name,
      String descriptor,
      String signature,
      Object value
  ) {
      if (fieldName.equals("removeMe")) {
            return null;
        }

      return super.visitField(access, name, descriptor, signature, value);
  }
}
```

Отфильтровываем ненужные поля

```java
public class Transformer {
    public static byte[] transform(byte[] originalClass) {
        ClassReader reader = new ClassReader(originalClass);
        ClassWriter writer = new ClassWriter(reader, 0);

        RemoveFieldClassVisitor visitor =
            new RemoveFieldClassVisitor(writer);

        reader.accept(visitor, 0);

        return writer.toByteArray();
    }
}
```

В итоге нехитрыми манипуляциями

логику «тяжёлой работы» можно перенести на build-time

Что мы можем делать с этим:

- Сканировать аннотации и классы
- Сканирование classpath
- Загрузка и валидация конфигураций
- Подготовка dependency injection
- Удаление лишнего

В итоге нехитрыми манипуляциями

логику «тяжёлой работы» можно перенести на build-time

Что мы можем делать с этим:

- Сканировать аннотации и классы
- Сканирование classpath
- Загрузка и валидация конфигураций
- Подготовка dependency injection
- Удаление лишнего
- **А главное: нужная кодогенерация!**

В итоге нехитрыми манипуляциями

логику «тяжёлой работы» можно перенести на build-time

Например:

```
field.setAccessible(true);
field.set(obj, dependency);
```



```
obj.field = dependency;
```

В итоге нехитрыми манипуляциями

логику «тяжёлой работы» можно перенести на build-time

Например:

```
field.setAccessible(true);
field.set(obj, dependency);
```



```
obj.field = dependency;
```

Напоминает компиляторные плагины
вроде All-open compiler plugin

ClassTransformingBuildStep

Quarkus делает подобные трансформации повсюду

ClassTransformingBuildStep: зачем?

Quarkus делает подобные трансформации повсюду



Hibernate (Panache), например

CDI

jakarta.enterprise.cdi

Азы и терминология

ArC — приблизительная имплементация спецификации Jakarta Contexts and Dependency Injection 4.1

Работает за счет ASM (того самого) и/или ClassFileApi

Contexts and Dependency Injection

```
@ApplicationScoped
public class UserService {
    public User findById(Long id) {
        return new User(id, "John");
    }
}
```

Singleton, с дополнительными
сгенерированными обертками и прокси

Contexts and Dependency Injection

```
// ===== Field или setter
@Inject
UserService userService;
```

И всё это хорошо ложится на build-time с инструментами, о которых мы говорили

ArC/Jakarta Interceptors

```
@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

@Fallback
```

ArC/Jakarta Interceptors

ArC во время build-time:

```
@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

@Fallback
```

ArC/Jakarta Interceptors

ArC во время build-time:

- Находит всё, что имеет аннотации-интерсепторы и т.д.

```
@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

@Fallback
```

ArC/Jakarta Interceptors

ArC во время build-time:

- Находит всё, что имеет аннотации-интерсепторы и т.д.
- Генерирует байткод, который не вызывает метод напрямую, а вставляет вызов цепочки интерсепторов

```
@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

@Fallback
```

ArC/Jakarta Interceptors

ArC во время build-time:

- Находит всё, что имеет аннотации-интерсепторы и т.д.
- Генерирует байткод, который не вызывает метод напрямую, а вставляет вызов цепочки интерсепторов
- В runtime, при вызове метода, сначала выполняются интерсепторы, потом и сам метод

```
@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

@Fallback
```

ArC/Jakarta Interceptors

```
public class MyService_Subclass extends MyService {

    @Override
    public void doWork() {
        // interceptor logic (например, транзакция)
        beginTransaction();
        try {
            super.doWork();
            commitTransaction();
        } catch (Exception e) {
            rollbackTransaction();
            throw e;
        }
    }
}
```

@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

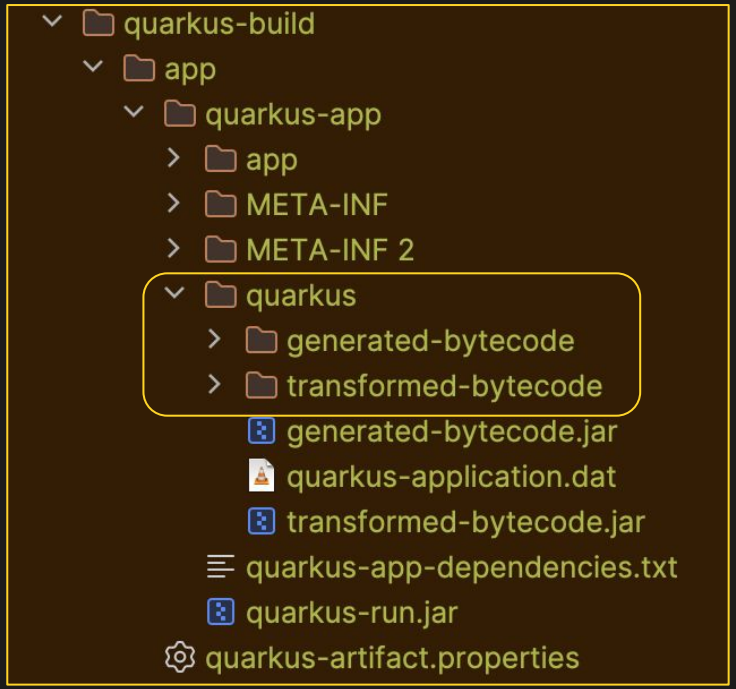
@Fallback

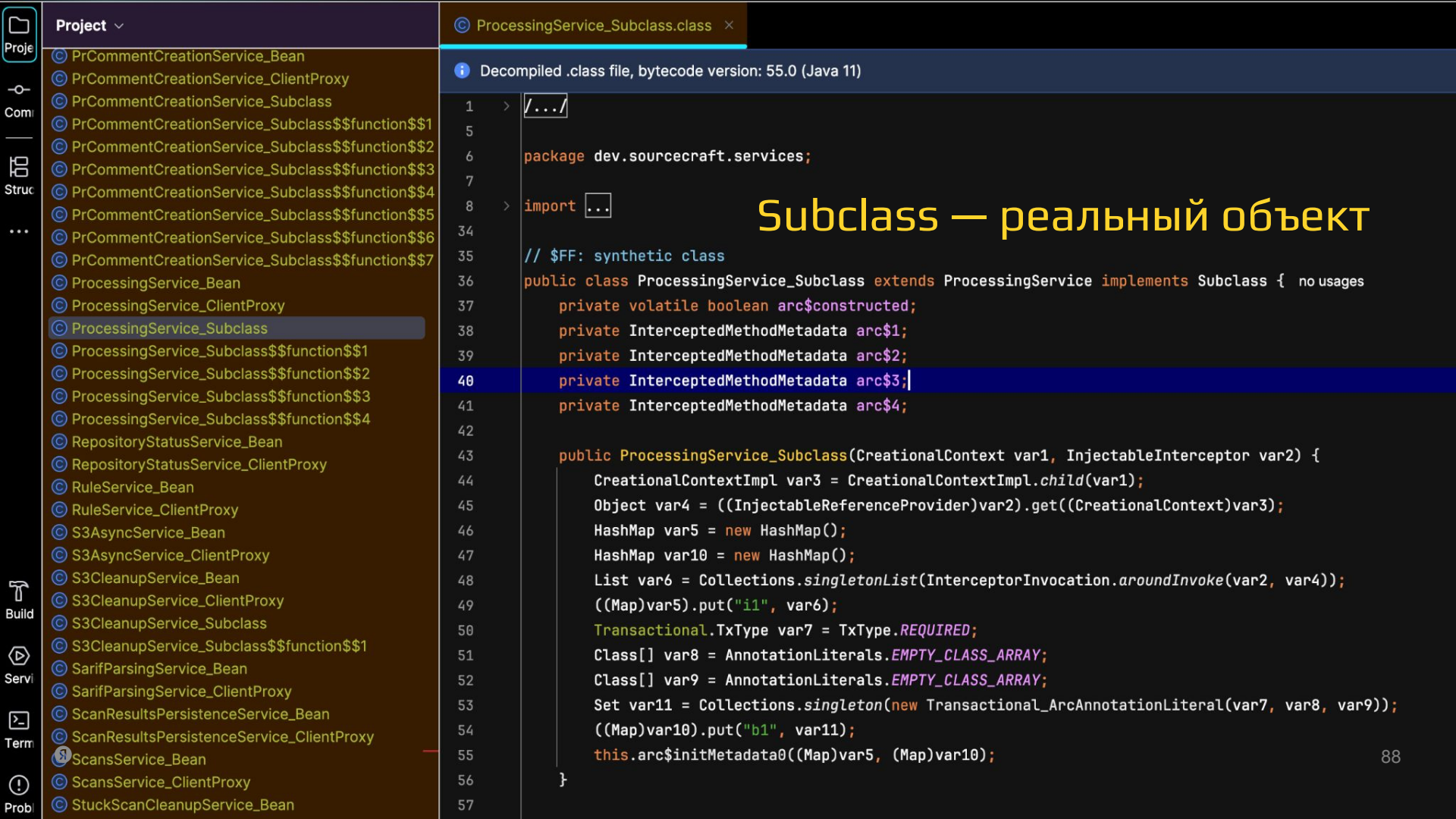
@InterceptorBinding

@RegisterInterceptor

Generated // transformed bytecode

```
@ApplicationScoped
class ProcessingService {
    fun persistResultToS3() {
        ...
    }
}
```




Project
PrCommentCreationService_Bean
PrCommentCreationService_ClientProxy
PrCommentCreationService_Subclass
PrCommentCreationService_Subclass$$function$$1
PrCommentCreationService_Subclass$$function$$2
PrCommentCreationService_Subclass$$function$$3
PrCommentCreationService_Subclass$$function$$4
PrCommentCreationService_Subclass$$function$$5
PrCommentCreationService_Subclass$$function$$6
PrCommentCreationService_Subclass$$function$$7
ProcessingService_Bean
ProcessingService_ClientProxy
ProcessingService_Subclass
ProcessingService_Subclass$$function$$1
ProcessingService_Subclass$$function$$2
ProcessingService_Subclass$$function$$3
ProcessingService_Subclass$$function$$4
RepositoryStatusService_Bean
RepositoryStatusService_ClientProxy
RuleService_Bean
RuleService_ClientProxy
S3AsyncService_Bean
S3AsyncService_ClientProxy
S3CleanupService_Bean
S3CleanupService_ClientProxy
S3CleanupService_Subclass
S3CleanupService_Subclass$$function$$1
SarifParsingService_Bean
SarifParsingService_ClientProxy
ScanResultsPersistenceService_Bean
ScanResultsPersistenceService_ClientProxy
ScansService_Bean
ScansService_ClientProxy
StuckScanCleanupService_Bean
ProcessingService_Subclass.class
Decompiled .class file, bytecode version: 55.0 (Java 11)
/.../
package dev.sourcecraft.services;
import ...
Subclass — реальный объект
// $FF: synthetic class
public class ProcessingService_Subclass extends ProcessingService implements Subclass { no usages
private volatile boolean arc$constructed;
private InterceptedMethodMetadata arc$1;
private InterceptedMethodMetadata arc$2;
private InterceptedMethodMetadata arc$3;
private InterceptedMethodMetadata arc$4;
public ProcessingService_Subclass(CreationalContext var1, InjectableInterceptor var2) {
CreationalContextImpl var3 = CreationalContextImpl.child(var1);
Object var4 = ((InjectableReferenceProvider)var2).get((CreationalContext)var3);
HashMap var5 = new HashMap();
HashMap var10 = new HashMap();
List var6 = Collections.singletonList(InterceptorInvocation.aroundInvoke(var2, var4));
((Map)var5).put("i1", var6);
Transactional.TxType var7 = TxType.REQUIRED;
Class[] var8 = AnnotationLiterals.EMPTY_CLASS_ARRAY;
Class[] var9 = AnnotationLiterals.EMPTY_CLASS_ARRAY;
Set var11 = Collections.singleton(new Transactional_ArcAnnotationLiteral(var7, var8, var9));
((Map)var10).put("b1", var11);
this.arc$initMetadata0((Map)var5, (Map)var10);
}
88

ProcessingService_Subclass.class

Decompiled .class file, bytecode version: 55.0 (Java 11)

persistResultToS3 2/5

```
36  public class ProcessingService_Subclass extends ProcessingService implements Subclass {  no usages
141
142     public String persistResultToS3(String var1, Commit var2, EngineDescription var3, ReportType var4, Scan var5) {
143         Object[] var6 = new Object[]{var1, var2, var3, var4, var5};
144         if (this.arc$constructed) {
145             try {
146                 InterceptedMethodMetadata var7 = this.arc$3;
147                 return (String)InvocationContexts.performAroundInvoke( target: this, var6, var7);
148             } catch (RuntimeException var9) {
149                 throw (Throwable)var9;
150             }
151         } else {
152             return this.persistResultToS3$$superforward(var1, var2, var3, var4, var5);
153         }
154     }
155
156     public void persistResultsToDb(Set var1, String var2) {
157         Object[] var3 = new Object[]{var1, var2};
158         if (this.arc$constructed) {
159             try {
```

Subclass — интерсепторы в логике

ProcessingService_Subclass.class | ProcessingService_Bean.class

Decompiled .class file, bytecode version: 55.0 (Java 11)

Bean-конструктор

```
26  public class ProcessingService_Bean implements InjectableBean, Supplier {  no usages
78
79      private ProcessingService doCreate(CreationalContext var1) {
80          Object var2 = this.interceptorProviderSupplier10.get();
81          ProcessingService var5 = (ProcessingService)(new ProcessingService_Subclass(var1, (InjectableInterceptor)var2
82
83          try {
84              Object var3 = this.injectProviderSupplier1.get();
85              CreationalContextImpl var4 = CreationalContextImpl.child((InjectableReferenceProvider)var3, var1);
86              Object var6 = ((InjectableReferenceProvider)var3).get((CreationalContext)var4);
87              var5.platformService = (PlatformService)var6;
88          } catch (RuntimeException var48) {
89              throw (Throwable)(new RuntimeException("Error injecting dev.sourcecraft.services.PlatformService dev.sour
90          }
91
92          try {
93              Object var8 = this.injectProviderSupplier2.get();
94              CreationalContextImpl var9 = CreationalContextImpl.child((InjectableReferenceProvider)var8, var1);
95              Object var10 = ((InjectableReferenceProvider)var8).get((CreationalContext)var9);
96              var5.repository4Artifact = (Repository4Artifact)var10;
97          } catch (RuntimeException var47) {
98              throw (Throwable)(new RuntimeException("Error injecting dev.sourcecraft.repositories.Repository4Artifact
99          }
```

@Unremovable

на случай, если
ArC будет шалить

ArcProcessor.generateResources()

на случай, если
вы вдруг захотите
шалить

Gizmo – обертка над манипуляциями с BC

```
Gizmo gizmo =
    Gizmo.create(ClassOutput.fileWriter(Path.of("target")));

gizmo.class_("com.example.Hello", cc → {
        cc.defaultConstructor();
        cc.staticMethod("main", mc → {
        ParamVar args = mc.parameter("args", String[].class);
            mc.body(bc → {
                bc.printf("Hello, World!%n");
                    bc.return_();
            });
        });
});
```

Gizmo 1 vs Gizmo 2

First off, Gizmo 1 is based on ASM and Gizmo 2 is based on the ClassFile API (not the one present in the JDK since version 24, but the fork maintained by David Lloyd, which supports Java 17). The ClassFile API itself is very different to ASM, and since the ClassFile API structure guided the Gizmo 2 API structure, that is also very different.



Что напоминает?

CrossConf 2024

Десериализатор

```
public class $serializer implements GeneratedSerializer {

    @NotNull
    public final A deserialize(@NotNull Decoder decoder) {
      ...
        while(var3) {
          int var4 = var7.decodeElementIndex(var2);
          switch (var4) {
            case -1:
              var3 = false;
              break;
            case 0:
              var6 = var7.decodeIntElement(var2, 0);
              var5 |= 1;
              break;
            default:
              throw new UnknownFieldException(var4);
          }
        }
      ...
```

Разбираем наше поле **b** как **Int**

84



С точной генерацией, подготовкой IR заранее и уходом от reflection понятно, а что еще бывает?

META-INF/jandex.idx

Очередной трюк!

META-INF/jandex.idx

Очередной трюк!

Jandex — это предрассчитанный индекс всех аннотаций и типов в classpath, использующийся для кодогенерации. А иногда и может использоваться на runtime.

META-INF/jandex.idx



Где еще пытаются AOT-like достичь оптимизаций в Java-мире?

- Spring AOT
- Java 24+ (AOT cache)
- GraalVM (действительно AOT)

483: Ahead-of-Time Class Loading & Linking

Spring PetClinic (21,000 classes):

Startup time reduced from

4.48s → 2.60s (42% improvement) Java 24

4.48s → 3.00s (33% improvement) Java 13+, AppCDS

39



Сохранение результатов JIT-компиляции (включая profiling-информацию) после training run.

Что из последнего?

JEP 516: Ahead-of-Time Object Caching with Any GC

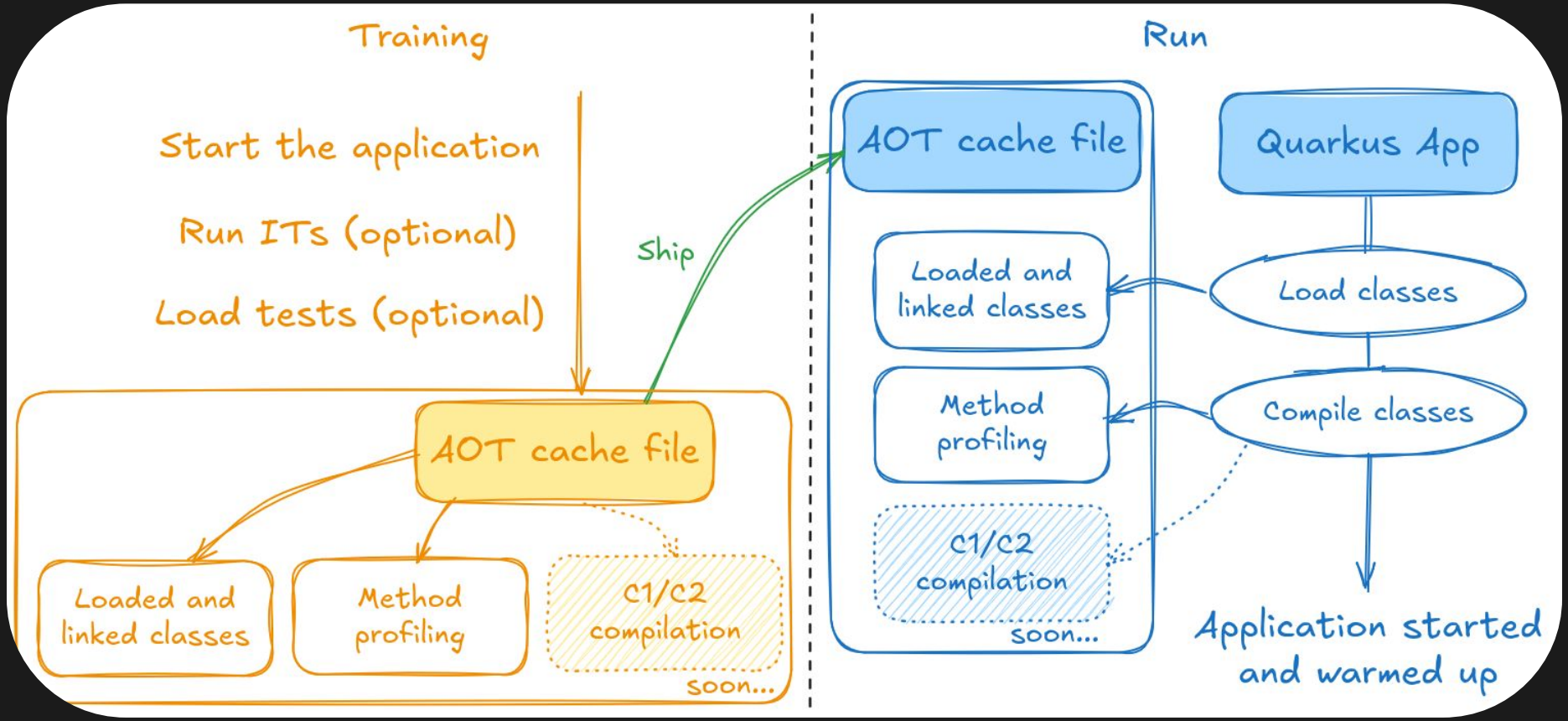


Enhance the ahead-of-time cache, which enables the HotSpot Java Virtual Machine to improve startup and warmup time, so that it can be used with any garbage collector, including the low-latency Z Garbage Collector (ZGC).

February 26, 2026 #release

Quarkus 3.32 - Project Leyden integration, more graceful shutdown, automatic Consul registration and more!

By  Guillaume Smet

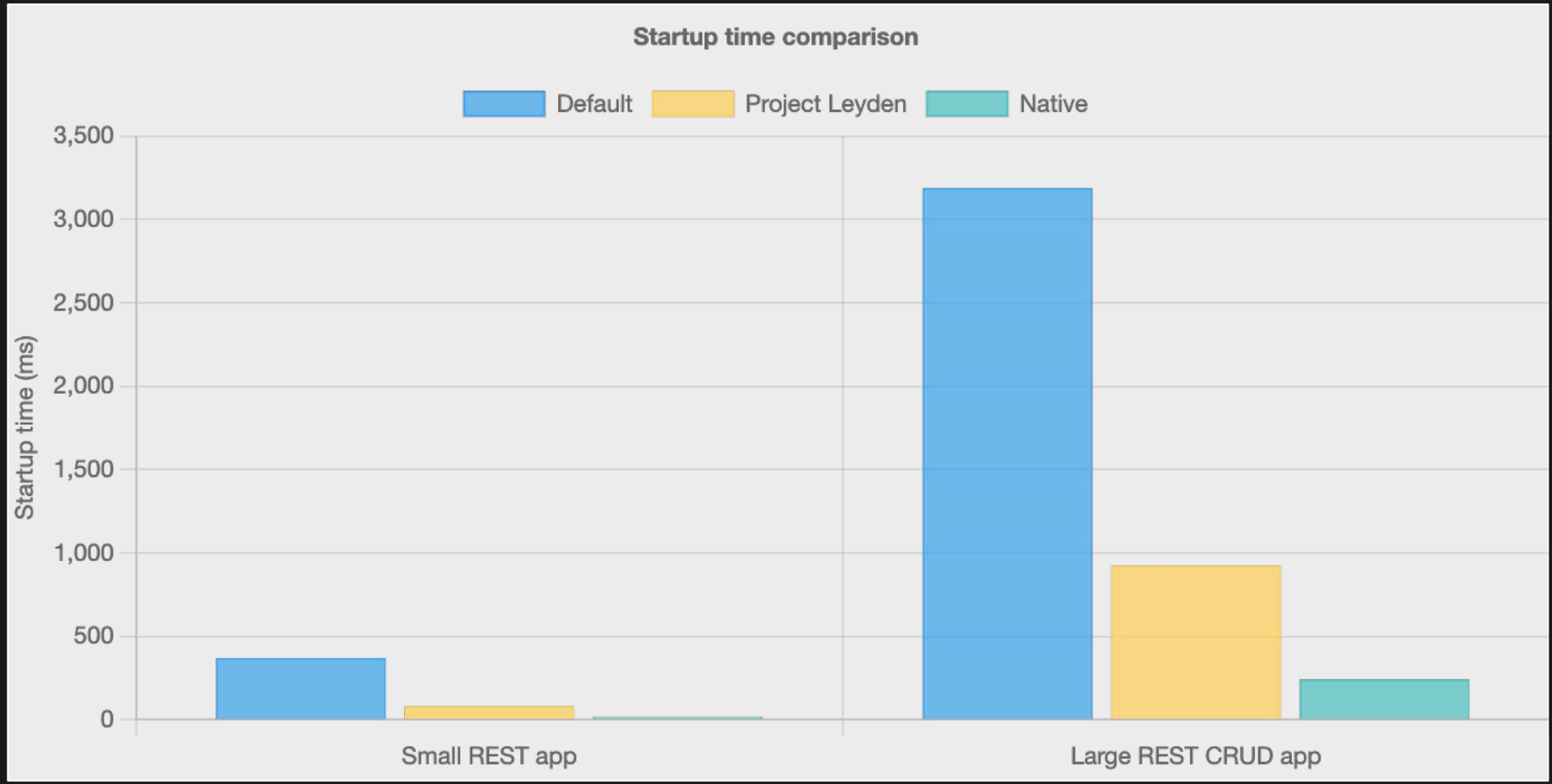

Training
Start the application
Run ITs (optional)
Load tests (optional)
Ship
AOT cache file
Loaded and linked classes
Method profiling
C1/C2 compilation
soon...
Run
AOT cache file
Quarkus App
Loaded and linked classes
Load classes
Method profiling
Compile classes
C1/C2 compilation
soon...
Application started and warmed up

- `./gradle build -Dquarkus.package.jar.aot.enabled=true`
- `./gradlew build quarkusIntTest buildAotEnhancedImage -Dquarkus.package.jar.aot.enabled=true`

https://quarkus.io/guides/aot

- Quarkus делает сборку
- Интеграционные тесты (`@QuarkusIntegrationTest`) тренируют AOT-cache
- JVM собирает данные профилятора
- `app.aot` cache сгенерирован
- ???
- PROFIT!

Страшные цифры!


Startup time comparison
Default
Project Leyden
Native
3,500
3,000
2,500
2,000
1,500
1,000
500
0
Startup time (ms)
Small REST app
Large REST CRUD app

Выводы

Выводы

- Выиграть в производительности можно ДАЖЕ там, где годами существуют подходы и решения

Выводы

- Выиграть в производительности можно ДАЖЕ там, где годами существуют подходы и решения
- Переносом бОльшей части логики на build time (например, DI)

Выводы

- Выиграть в производительности можно ДАЖЕ там, где годами существуют подходы и решения
- Переносом бОльшей части логики на build time (например, DI)
- Игры с AOT или “псевдо”-AOT – это тренд

Выводы



Яндекс

Спасибо!
Q&A

Андрей Кулешов

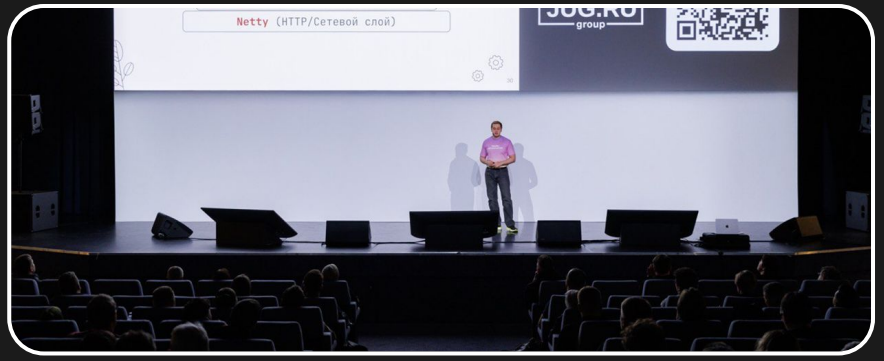


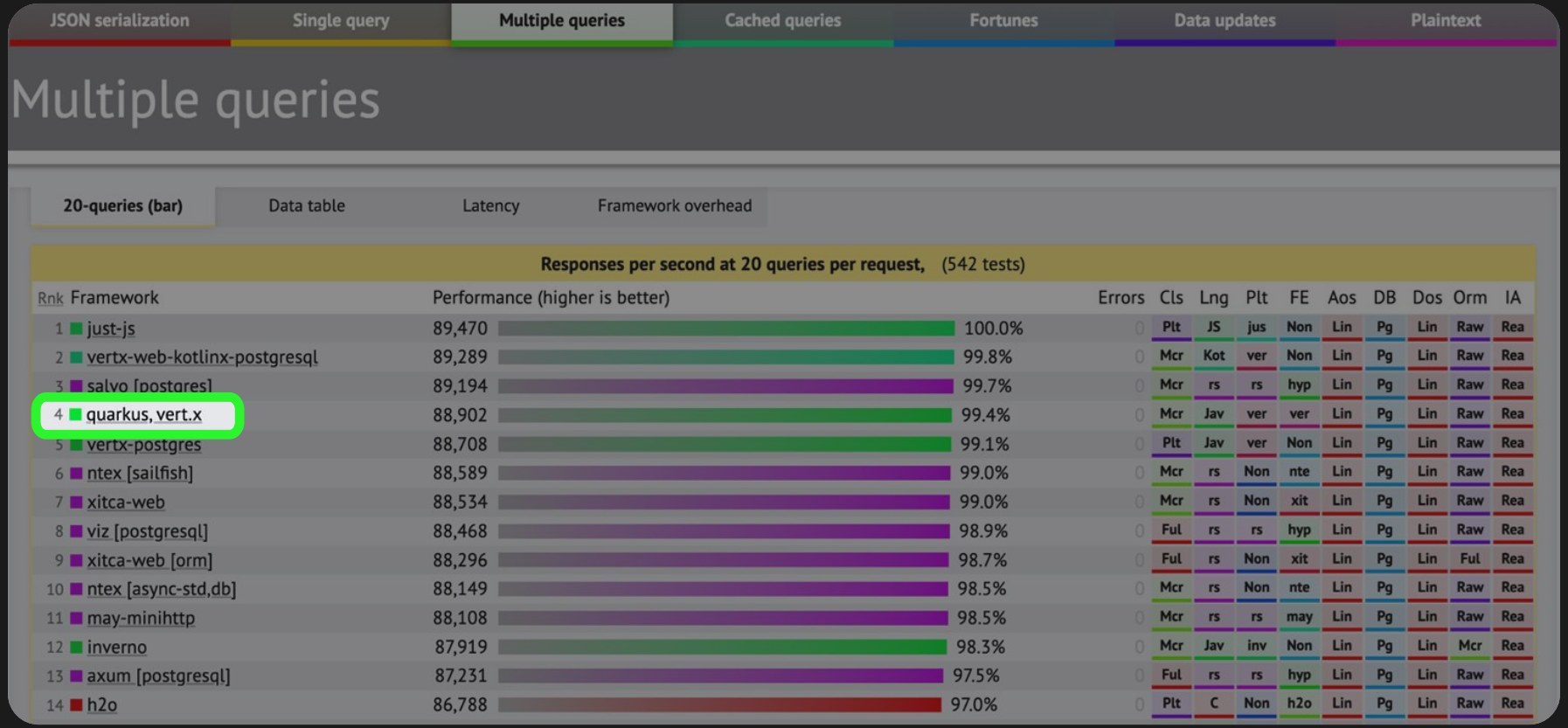
В предыдущих сериях



Не Spring'ом единым?
Смотрим на Quarkus







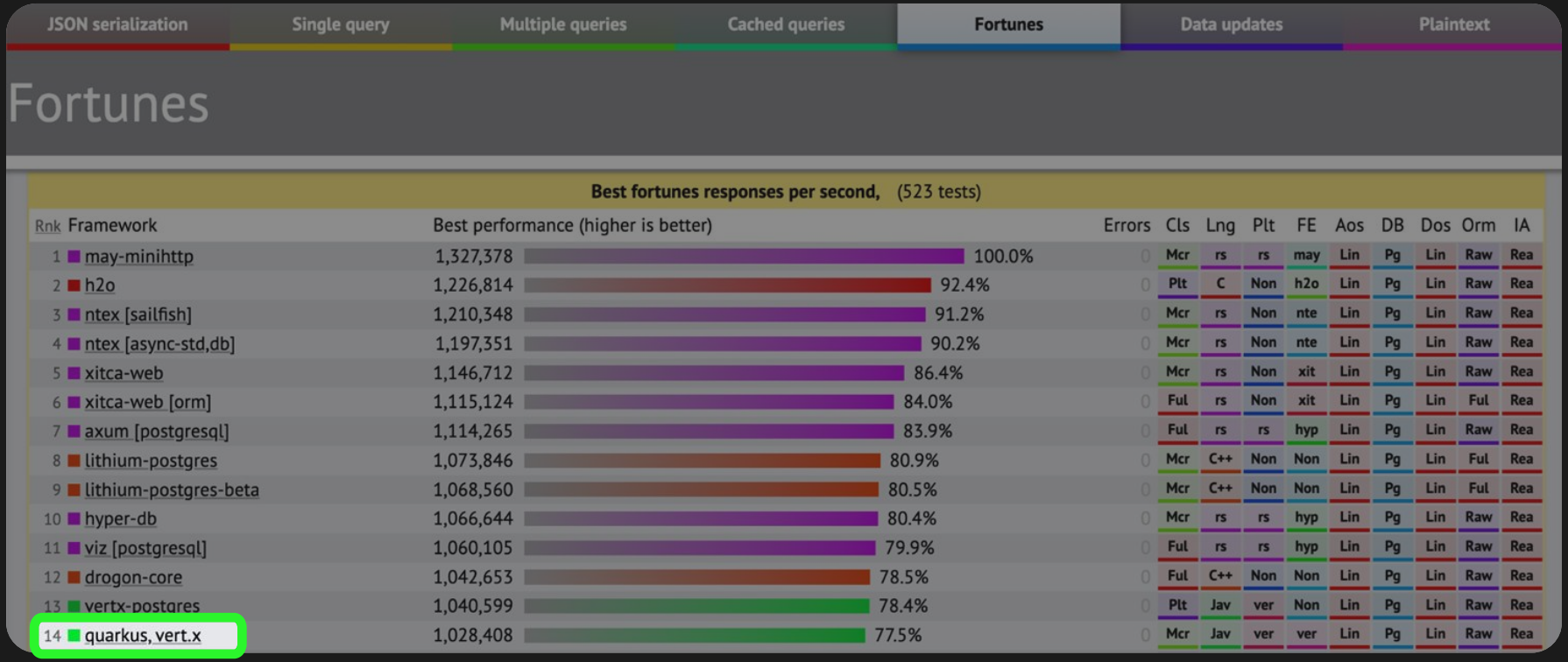
JSON serialization | Single query | Multiple queries | Cached queries | Fortunes | Data updates | Plaintext

Multiple queries

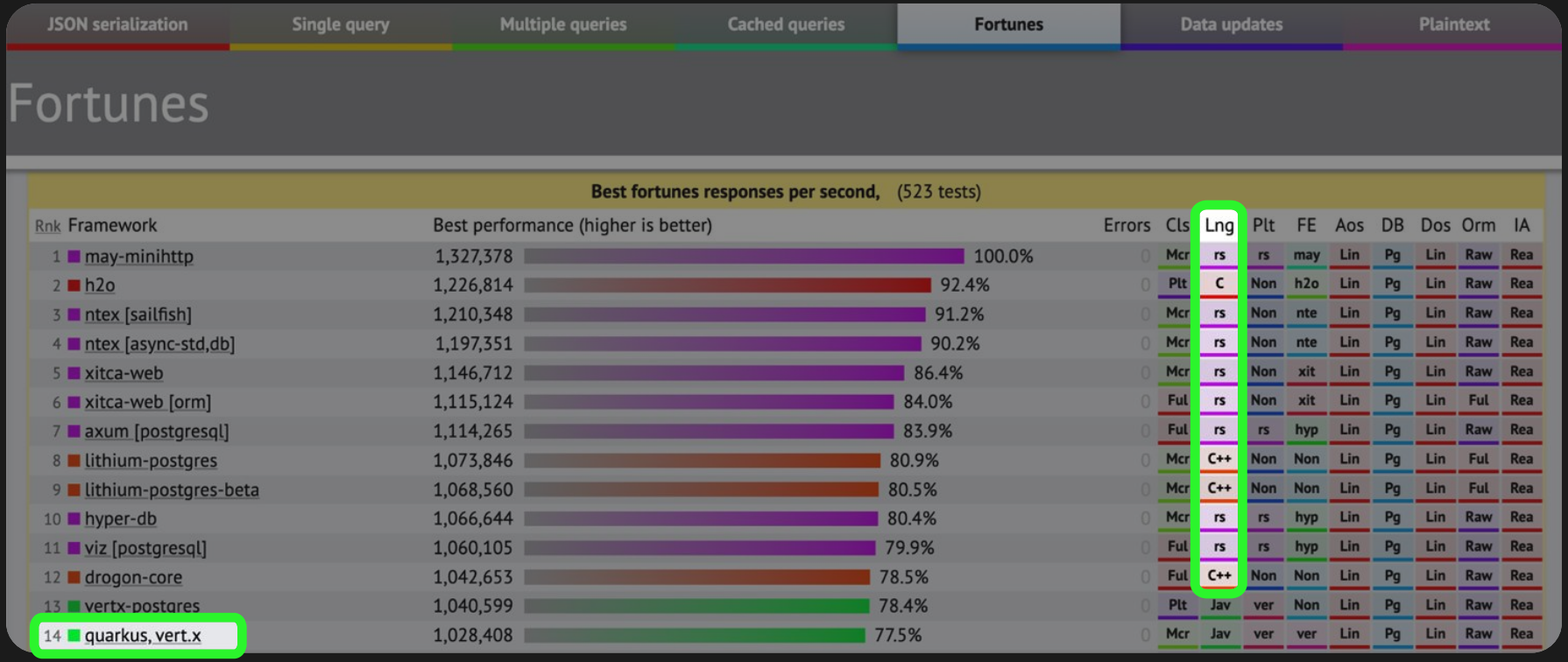
20-queries (bar) | Data table | Latency | Framework overhead

Responses per second at 20 queries per request, (542 tests)

Rnk	Framework	Performance (higher is better)		Errors	Cls	Lng	Plt	FE	Aos	DB	Dos	Orm	IA
1	just-js	89,470	100.0%	0	Plt	JS	jus	Non	Lin	Pg	Lin	Raw	Rea
2	vertx-web-kotlinx-postgresql	89,289	99.8%	0	Mcr	Kot	ver	Non	Lin	Pg	Lin	Raw	Rea
3	salvo [postgres]	89,194	99.7%	0	Mcr	rs	rs	hyp	Lin	Pg	Lin	Raw	Rea
4	quarkus, vert.x	88,902	99.4%	0	Mcr	Jav	ver	ver	Lin	Pg	Lin	Raw	Rea
5	vertx-postgres	88,708	99.1%	0	Plt	Jav	ver	Non	Lin	Pg	Lin	Raw	Rea
6	ntex [sailfish]	88,589	99.0%	0	Mcr	rs	Non	nte	Lin	Pg	Lin	Raw	Rea
7	xitca-web	88,534	99.0%	0	Mcr	rs	Non	xit	Lin	Pg	Lin	Raw	Rea
8	viz [postgresql]	88,468	98.9%	0	Ful	rs	rs	hyp	Lin	Pg	Lin	Raw	Rea
9	xitca-web [orm]	88,296	98.7%	0	Ful	rs	Non	xit	Lin	Pg	Lin	Ful	Rea
10	ntex [async-std,db]	88,149	98.5%	0	Mcr	rs	Non	nte	Lin	Pg	Lin	Raw	Rea
11	may-minihttp	88,108	98.5%	0	Mcr	rs	rs	may	Lin	Pg	Lin	Raw	Rea
12	inverno	87,919	98.3%	0	Mcr	Jav	inv	Non	Lin	Pg	Lin	Mcr	Rea
13	axum [postgresql]	87,231	97.5%	0	Ful	rs	rs	hyp	Lin	Pg	Lin	Raw	Rea
14	h2o	86,788	97.0%	0	Plt	C	Non	h2o	Lin	Pg	Lin	Raw	Rea



Fortunes – производительность по числу ответов в секунду

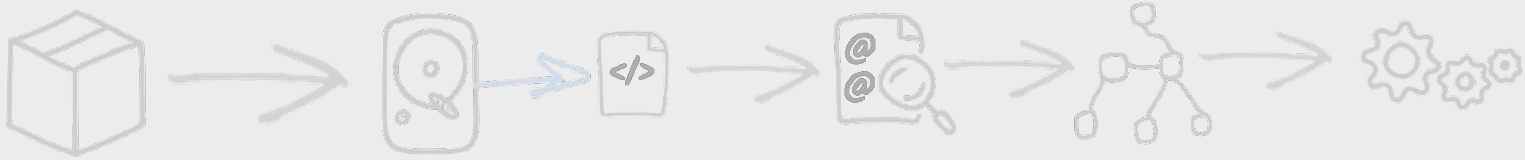


Нативные языки

Traditional Frameworks

Build Time

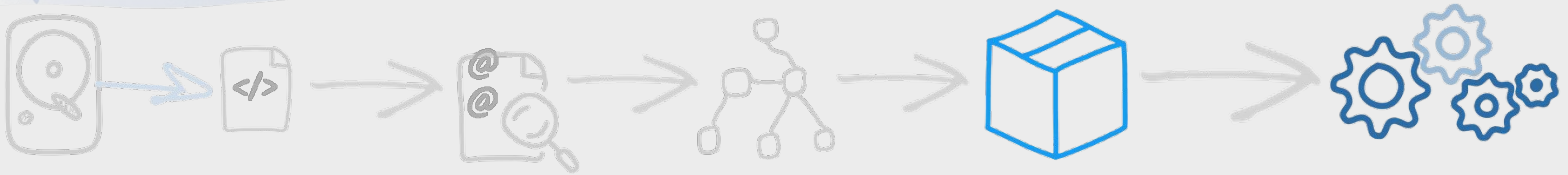
Runtime



Quarkus

Build Time

Runtime



Вспомним азы:

лексика, синтаксис, семантика

Лексика, семантика, синтаксис

```
fun fo@o() {
    return list0f(“Error”)
}
```

```
fun foo(0)(0) {
    return list0f(“Error”)
}
```

```
fun foo(): String {
    return list0f(“Error”)
}
```

Лексика, семантика, синтаксис

Лексический анализ

```
fun fo@o() {
    return list0f(“Error”)
}
```

Синтаксический анализ

```
fun foo(0)(0) {
    return list0f(“Error”)
}
```

Семантический анализ

```
fun foo(): String {
    return list0f(“Error”)
}
```

Яндекс

Спасибо!
Q&A

Андрей Кулешов



WHY LEARN EXTENSIONS COMMUNITY START CODING



Quarkus Performance

Designed for Fast Startup, High Throughput, and Low Resource Consumption



Quarkus is engineered to be efficient by using build-time optimizations and a reactive core to achieve fast startup times, high throughput, low response latency, reduced memory footprint, and minimal resource consumption. As a result, Quarkus is fast... real fast.

WHY LEARN EXTENSIONS COMMUNITY START CODING

Quarkus Performance

Designed for Fast Startup, High Throughput, and Low Resource Consumption



Quarkus is engineered to be efficient by using build-time optimizations and a reactive core to achieve fast startup times, high throughput, low response latency, reduced memory footprint, and minimal resource consumption. As a result, Quarkus is fast... real fast.

Contexts and Dependency Injection

```
// ===== Конструктор
private final PaymentService paymentService;

@Inject
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

Contexts and Dependency Injection

```
// ===== Метод
@Inject
void configure(ConfigService config, LogService log) {
    // Вызывается после создания объекта
}
```